PHOT 110: Introduction to programming

Collected exercises, Spring semester (2024-now)

Michaël Barbier

2026-03-25

Table of contents

Preface **3**

1 Expressions in the Python console **4**
1.1 Commando-style calculations . . . 4
1.2 Expressions using functions of the math package . . . 4

2 Input, formatting output, and expressions in Python scripts **5**
2.1 Body Mass Index calculator . . . 5
2.2 Text transformations . . . 5
2.3 Tsiolkovsky rocket equation . . . 5
2.4 Formatted table: unit conversion . . . 6
2.5 Glorifier (Spring 2025) . . . 6
2.6 Repeat words (Spring 2025) . . . 6
2.7 Braking distance (Spring 2025) . . . 6
2.8 Velocity measurement by radar gun (Spring 2025) . . . 7
2.9 Chevaux-Vapeur car tax calculator (Spring 2025) . . . 7
2.10 Formatted table of numbers (Spring 2025) . . . 7

3 Conditional Statements **8**
3.1 Check login . . . 8
3.2 Roots of a quadratic equation . . . 8

4 For and While loops **9**
4.1 Number series . . . 9
4.2 Power series . . . 9
4.3 Printing numbers (Spring 2025) . . . 9
4.4 Decreasing series (Spring 2025) . . . 10
4.5 Prime numbers . . . 10
4.6 Play rock-paper-scissors . . . 10
4.7 Game of higher-lower (Spring 2025) . . . 11

5 Lists **12**
5.1 Fibonacci sequence . . . 12
5.2 Printing elements of a list (Spring 2025) . . . 12
5.3 List of soccer players (Spring 2025) . . . 12
5.4 Selling fruit (Spring 2025) . . . 13

6 Turtle graphics **14**
6.1 Fibonacci square tiling with turtle . . . 14
6.2 Fibonacci sunflower pattern with turtle . . . 14
6.3 Draw random dots with turtle . . . 15

7 Sequences and operations: Lists, Tuples, Strings **17**
7.1 Making a tuple (Spring 2025) . . . 17
7.2 String chemistry (Spring 2025) . . . 17
7.3 Yoda-talk (Spring 2025) . . . 17

8 Arrays **19**
8.1 Create Numpy arrays (Spring 2025) . . . 19
8.2 Access elements in Numpy arrays (Spring 2025) . . . 19
8.3 Different ways to generate the x-values: with Numpy (Spring 2025) . . . 20
8.4 Element-wise operations with Numpy arrays (Spring 2025) . . . 20
8.5 Expressions with Numpy arrays . . . 21

9 Plotting graphs with Matplotlib **22**
9.1 Plot a Gaussian and Lorentz distribution (Spring 2025) . . . 24
9.2 Lissajoux figures (Spring 2024) . . . 25
9.3 Refraction at a glass interface (Spring 2024) . . . 26
9.4 Adapt previous plot to include axis labels, etc. (Spring 2024) . . . 27
9.5 Extend previous script to show multiple incident angles (Spring 2024) . . . 28
9.6 Plot ellipses with various parameters (Spring 2024) . . . 29
9.7 Plot Poinsot's spiral (2nd form) (Spring 2024) . . . 30
9.8 Plot a Gaussian distribution (Spring 2024) . . . 30
9.9 Make a scatter plot of a Gaussian distribution in 2D (Spring 2024) . . . 31
9.10 Adapt the scatter plot of previous exercise by specifying the size and color of the dots (Spring 2024) . . . 32
9.11 2D domains (Spring 2024) . . . 33
9.12 2D arrays (Spring 2024) . . . 33

10 Exercises on functions **35**
10.1 Calculate the factorial of a number (Spring 2024) . . . 35
10.2 Calculate the number of combinations (Spring 2024) . . . 35
10.3 Intersection of two lines (Spring 2024) . . . 36
10.4 Intersection of many lines (Spring 2024) . . . 36
10.5 Drawing the Koch curve (Spring 2025) . . . 37

11 Exercises on error handling **38**
11.1 User console input (Spring 2024) . . . 38
11.2 Read from a text file (Spring 2024) . . . 38

12 Python Custom Modules **39**
12.1 A simple module for calculations (Spring 2025) . . . 39
12.2 A module for light at an interface (Spring 2025) . . . 39
12.3 A module for converting text (Spring 2025) . . . 40

13 Dictionaries & data tables (Pandas dataframes) **41**
13.1 Creating a dictionary (Spring 2025) . . . 41
13.2 Convert to a module (Spring 2025) . . . 41
13.3 More advanced datasets: Pandas package . . . 41
13.4 Download a dataset . . . 41

14 Classes **43**
14.1 Library system . . . 43
14.2 Subclasses . . . 44

15 Numerical algorithms **45**
15.1 Compute an integral numerically (Spring 2024) . . . 45
15.2 Compute derivatives numerically (Spring 2024) . . . 46
15.3 Solve a system of equations . . . 47

Solutions **48**
Input, formatting output, and expressions in Python scripts . . . 48
2.1 Body Mass Index calculator [Spring, 2025-2026] . . . 48
2.2 Text transformations [Spring, 2025-2026] . . . 48
2.3 Tsiolkovsky rocket equation [Spring, 2025-2026] . . . 48
2.4 Formatted table: unit conversion [Spring, 2025-2026] . . . 48
2.5 Glorifier [Spring, 2024-2025] . . . 48
2.6 Repeat words [Spring, 2024-2025] . . . 48
2.7 Braking distance [Spring, 2024-2025] . . . 48
2.8 Velocity measurement by radar gun [Spring, 2024-2025] . . . 49
2.9 Chevaux-Vapeur car tax calculator [Spring, 2024-2025] . . . 49
2.10 Formatted table of numbers [Spring, 2024-2025] . . . 49
Conditional Statements . . . 49
3.1 Check login [Spring, 2025-2026] . . . 49
3.2 Roots of a quadratic equation [Spring, 2025-2026] . . . 49
For and While loops . . . 49
4.1 Number series [Spring, 2025-2026] . . . 49
4.2 Power series [Spring, 2025-2026] . . . 49
4.3 Printing numbers [Spring, 2024-2025] . . . 49
4.4 Decreasing series [Spring, 2024-2025] . . . 50
4.5 Prime numbers [Spring, 2025-2026] . . . 50
4.6 Play rock-paper-scissors [Spring, 2025-2026] . . . 50
4.7 Game of higher-lower [Spring, 2024-2025] . . . 50
Lists . . . 50
5.1 Fibonacci sequence . . . 50
5.2 Printing elements of a list (Spring 2025) . . . 50
5.3 List of soccer players (Spring 2025) . . . 51
5.4 Selling fruit (Spring 2025) . . . 51

Preface

This document contains the exercises of the PHOT110 - Introduction to Programming course, given at Izmir Institute of Technology to the students of the Photonics Department. The solutions for the problems can be found at the end of the book.

For other course materials of the PHOT110 course please visit https://web.iyte.edu.tr/michaelbarbier/education_phot110.html.

Knuth (1984)

1 Expressions in the Python console

1.1 Commando-style calculations

Calculate the final value for y for each the three columns, start at the top where x is defined, then perform the assignments in each next row.

$x = 4$	$x = 20$	$x = -5$
$y = x^2$	$y = x/4$	$y = x^{(x+3)}$
$y = 5 \times y$	$y = y + 2$	$y = \dfrac{y}{5-y}$
$y = y - 23$	$y = 3/2\, y$	$y = 4.5\, y$
$y = \dfrac{y}{2}$	$y = \sqrt{y} = y^{0.5}$	$y = \dfrac{1}{y}$
$y =$?	$y =$?	$y =$?

1.2 Expressions using functions of the math package

Calculate the values y of following expressions, where $a = 3$, $b = 3/7$, $c = 3.00 \times 10^8$. First assign the parameters a, b, c, and import the functions you need from the `math` package. For the complex exponential function import `exp` from `cmath` instead of `math`. To use the imaginary unit i type `1j`. The results are given in blue so you can compare:

$y = 3a^5 - b^2 \quad \approx 728.8$	$y = \dfrac{iab}{a-b} \quad = 0.5i$
$y = (40/c)^4 \quad \approx 3.16 \times 10^{-28}$	$y = \sqrt{5}/b^2 \quad \approx 12.2$
$y = \sin(b\pi) \times \dfrac{1+b^2}{a^2} \quad \approx 0.13$	$y = \arctan(a) \quad \approx 1.25$
$y = \dfrac{\sinh(1)}{2} \quad \approx 0.59$	$y = \dfrac{e^{i\pi/4}}{\sqrt{2}} \quad = \dfrac{1+i}{2}$
$y = \dfrac{6!}{10!} \quad \approx 1.98 \times 10^{-4}$	$y = e^{ia\pi} \quad = -1$

More expressions, put $x = 4$, $a = 2$, $b = 10$ and calculate the following expressions:

$y = 4\,\dfrac{a\,x}{1+\sin(x)}$	$y = \sinh(a\,x)$
$y = \tan(x^2)$	$y = x/\mathrm{erf}(x/b)$
$y = 5 \times \exp(\sin(b\,x)/x^a)$	$y = x + 2\,\sqrt{a - x/b}$
$y = \sqrt{ix + a}$	$y = \cos(2\pi\, a\, x)$
$y = \dfrac{x^{ia+1}}{b}$	$y = \cos(x) \times \sqrt{\sin(x)}$

2 Input, formatting output, and expressions in Python scripts

2.1 Body Mass Index calculator

Prompt a user for his/her height and length, then calculate his/her body mass index and print it to the screen. The body mass index is a rather inaccurate measure for body fat and is given by the formula: BMI = weight/height^{2}kg/m^2 where the weight is in kilograms and height in meters.

```
Height (in cm): 179
Weight (in kg): 75
Your BMI = 23.41
```

2.2 Text transformations

Prompt the user for a sentence. The perform the following tasks:

a) Make all characters in the sentence upper case.
b) Surround the sentence by double quotes.
c) Append a question mark "?" to the sentence and print it n times on the same line (where n is a parameter).
d) Print the words of the sentence on separate lines by replacing spaces by `\n` as a line separator (`\n` is the escape code to start a new line)

```
Please provide a sentence: Apples are sometimes green
(a) Uppercase sentence: APPLES ARE SOMETIMES GREEN
(b) With quotes: "Apples are sometimes green"
(c) Repeated 4 times: Apples are sometimes green? Apples are sometimes green? Apples are somet
(d) On separate lines:
Apples
are
sometimes
green
```

2.3 Tsiolkovsky rocket equation

To launch a satelite into a Lower Earth Orbit (LEO) a space rocket needs to reach an orbit velocity Δv of approximately $\Delta v \approx 7.8$ km/s or $28,080$ km/h. The amount of fuel required depends on the mass of the empty rocket plus satelite and fuel via the Tsiolkovsky rocket equation:

$$\Delta v = v_e \ln \frac{m_0}{m_f} \qquad \Rightarrow \qquad \frac{m_0}{m_f} = \frac{m_{\text{rocket}} + m_{\text{satelite}} + m_{\text{fuel}}}{m_{\text{rocket}} + m_{\text{satelite}}} = e^{\Delta v / v_e}$$

Where m_0 is the initial mass of the rocket (and satelite) including propellant (fuel), m_f is the final mass of the empty rocket (with satelite), and $v_e = 2.5$ km/s the effective exhaust velocity for this specific rocket.

Prompt the user to give a masses of the satelite and the empty rocket to be launched (m_f) and respond with the required fuel to reach the required $\Delta v = 7.8$ km/s.

2.4 Formatted table: unit conversion

Prompt the user for a distance in meters. Then print a table with the distance in units of inches, feet, yard, and mile on the different rows. The units can be converted to metric units by: 1 inch = 2.54 cm, 1 foot = 12 inches, 1 yard = 3 feet, and 1 mile = 1,760 yards. Column 1 should represent the distance with two digits after the decimal point, in column 2 with scientific notation with 2 significant figures, and the last column the units. Use formatted strings to make sure the columns are equally wide.

2.5 Glorifier (Spring 2025)

Prompt a user for his/her name. Print the sentence: "All hail [name of the user]" as a response.

2.6 Repeat words (Spring 2025)

Prompt the user for a word. Print the word 8 times on one line with spaces in between and put an exclamation mark at the end.

2.7 Braking distance (Spring 2025)

Prompt a user for the speed of a car. Calculate the braking distance on asfalt and concrete roads and print them. The (kinetic) friction constants for concrete and asphalt are given in following table:

Material	Kinetic	Static
Rubber on concrete (dry)	0.68	0.90
Rubber on concrete (wet)	0.58	-.–
Rubber on asphalt (dry)	0.72	0.68
Rubber on asphalt (wet)	0.53	-.–
Rubber on ice	0.15	0.15

The braking distance D for a car driving at a speed v is determined by the perception-reaction time $t_r \approx 1$ of the driver and the friction coefficient of the road μ according to the following formula ($g = 9.81\, m/s^2$):

$$D = vt_r + \frac{v^2}{2\mu g}$$

2.8 Velocity measurement by radar gun (Spring 2025)

Traffic police officers can use a radar gun to measure the speed of a car. The difference in frequency f_t send out by the radar gun and the received echo f_e gives the Doppler frequency $f_d = f_t - f_e$. This frequency is related to the velocity of the car by:

$$f_d \approx 2v\frac{f_t}{c'}$$

where $c' = c/1.0003$. Suppose the radar device is operated in the K-band: $f_t = 24$ GHz. Calculate the Doppler beating frequency f_d measured by the officer if you pass him with a speed $v = 200$ km/h.

2.9 Chevaux-Vapeur car tax calculator (Spring 2025)

Automatically calculate the tax on cars in France (before 1957). For this, ask a user following properties of his/her car:

- Cylinder capacity of his/her car C (e.g. 1.3 liters)
- Maximum revolutions/minute ω

Then print the amount that the user should pay for his/her car. To calculate the tax use the following formula:

$$CV = CK(60\omega)$$

where K is for a four-cylinder engine: 1.5×10^{-4}.

2.10 Formatted table of numbers (Spring 2025)

Prompt the user for two floating point numbers and print a table with on the rows the numbers the number itself, the square, and the square root of the number and as columns the two resulting numbers and their sum. Use formatted strings to set the number of digits after the decimal point.

3 Conditional Statements

3.1 Check login

Prompt the user to give his/her username and password. Verify whether the username is `amapirate` and his/her password is `greenbeans`. If correct, tell the user that he the login was successful. Example output:

```
Please provide your login credentials:
username: Micky
password: redbeans
Sorry, your username or password is incorrect.
```

3.2 Roots of a quadratic equation

The roots of a quadratic equation: $ax^2 + bx + c = 0$ are given by:

$$x_{\pm} = \frac{-b \pm \sqrt{D}}{2a} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}, \qquad \text{with discriminant } D = b^2 - 4ac$$

The number of roots is determined by the value of the discriminant D:

- $D > 0$: two roots, x_- and x_+,
- $D = 0$: one root, $x = x_- = x_+$,
- $D < 0$: no real roots, x_- and x_+ become complex.

Prompt the user for parameters a, b, and c of the quadratic equation, then calculate the number of roots by calculating first the discriminant $D = b^2 - 4ac$. Afterwards print the (real-valued) roots. Example output:

```
--- Roots of the equation ax^2 + bx + c = 0 ---
Parameter a: 2
Parameter b: 3
Parameter c: -5
The discriminant D = 49 > 0, two roots x = -2.5, 1.0
```

4 For and While loops

4.1 Number series

Print the values of $1/n$ for $n \in [5, 10]$, each on a separate line. Use first a while loop, then try to use a for loop (which one is easier?). The output should look as follows:

```
1/5 = 0.2
1/6 = 0.167
1/7 = 0.143
1/8 = 0.125
1/9 = 0.111
1/10 = 0.1
```

4.2 Power series

Consider the approximation of Eulers constant $e = \sum_{n=0}^{\infty} \frac{1}{n!}$. Print the consecutive approximations where you only take the first $1, 2, 3, 4, ...$ terms into account and print each of them on a separate line to the console. Stop when the approximation only differs by 0.001 or less from the real value of $e = 2.7183$ (make use of `math.e`, `math.factorial()`, and `math.fabs()`). The output should look as follows:

```
Euler's e = 2.7183 approximated to 1 terms =   1.0
Euler's e = 2.7183 approximated to 2 terms =   2.0
Euler's e = 2.7183 approximated to 3 terms =   2.5
Euler's e = 2.7183 approximated to 4 terms = 2.6667
Euler's e = 2.7183 approximated to 5 terms = 2.7083
Euler's e = 2.7183 approximated to 6 terms = 2.7167
Euler's e = 2.7183 approximated to 7 terms = 2.7181
```

4.3 Printing numbers (Spring 2025)

- Use a while loop to print the numbers from 1 to 10 to the screen.
- Then use a for loop for the same task. Make use of the `range()` function for this.
- Finally, use a `for` loop to print the odd numbers from 1 to 10 to the screen.

As an example, the output of the last part should give:

```
Print odd numbers 1-10 using a for loop
1
3
5
7
9
```

4.4 Decreasing series (Spring 2025)

- Use a `while` loop to print the terms of series $\sum_{n=1}^{\infty} \frac{1}{n^4}$ for which $\frac{1}{n^4} \geq 0.0001$.
- Repeat the same task but using a `for` loop (combine with a conditional statement). Remark that here the lazy evaluation of the `range()` function is also a benefit.

Output of either script should have following output:

```
1.0
0.0625
0.012345679012345678
0.00390625
0.0016
0.0007716049382716049
0.00041649312786339027
0.000244140625
0.00015241579027587258
```

4.5 Prime numbers

Prompt the user for a positive integer number n. Verify whether this number is prime (does not have any integer divisors except 1 and n). For this you can loop over all numbers $m \leq \sqrt{n}$ and check whether n/m is a natural number. Use `math.sqrt()` for the square root and the modulo operator `%` which gives the rest after division (e.g. `13 % 5 = 3`) to verify whether n can be divided by $m = 2, 3, 4, \ldots$. The output should look as follows:

```
Give a positive integer n: 169
The number 169 can be divided by 13
The number 169 is not prime
```

4.6 Play rock-paper-scissors

Prompt the player repetitively to choose from: [1] Rock , [2] Paper, [3] Scissors, or [4] Stop the game. Award points for winning (+1), losing (-1), or a draw (0). Use the function `randint(1, 3)` from package `random` to obtain random integers from the set $\{1, 2, 3\}$. Stop the script when the choice was 4. Example output:

```
--- Let's play Rock-Paper-Scissors ---
  [1] Rock , [2] Paper, [3] Scissors, or [4] Stop the game
Choose 1-4: 3
I had the same, score: 0

  [1] Rock , [2] Paper, [3] Scissors, or [4] Stop the game
Choose 1-4: 2
You lose, I had Scissors, score: -1

  [1] Rock , [2] Paper, [3] Scissors, or [4] Stop the game
Choose 1-4: 1
You win, I had Scissors, score: 0
```

```
  [1] Rock , [2] Paper, [3] Scissors, or [4] Stop the game
Choose 1-4: 4

Stopping game!
```

4.7 Game of higher-lower (Spring 2025)

Create a simple version of the higher-lower game where a person thinks of a number and the other has to guess it. Generate a random number using following code:

```
import random
n = random.randint(0, 100)
```

Afterwards, prompt the user repetitively to make a guess, and tell each time whether the number is higher or lower than the guess. If the user guesses the actual number, stop the game. See following example output of a correct working script:

```
Guess the number [0, 100]: game started!
What is the number? 45
The number is lower, try again.
What is the number? 10
The number is lower, try again.
What is the number? 4
The number is higher, try again.
What is the number? 7
Yes, that is the number! Congratulations, you guessed it in 4 guesses.
```

5 Lists

5.1 Fibonacci sequence

Calculate the first 10 numbers of the Fibonacci sequence, put them in a list and print the list to the screen. The Fibonacci sequence is defined as follows: $F_0 = 0$, $F_1 = 1$, and $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$. More information about the Fibonacci sequence can be found on Wikipedia.

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

5.2 Printing elements of a list (Spring 2025)

Create the following list:

["yellow", "green", "blue", "red", "white", "orange", "black"]

a. Print the last element to the screen
b. Print the sublist containing the colors of the Turkish flag
c. Print the sublist with the colors of the rainbow (ignore black and white) and order them according to increasing wavelengths
d. Print all the elements one by one to the screen using a loop.
e. Prompt the user to tell his favorite color which is not in the list. If not in the list, append it to the list. Otherwise reply that it is already in the list.

Example output for tasks (a) and (b):

```
Last color in the list:
black
List of colors of the Turkish flag:
['red', 'white']
```

5.3 List of soccer players (Spring 2025)

Define a list containing the names of some of the soccer players from the Kazakhstan national football team (men):

["Maksim", "Abat", "Oralkhan", "Ramazan", "Vyacheslav"]

Then perform the following tasks:

- Print the names of the list to the screen line-by-line by using a for loop.
- Print the names in reversed order to the screen. Hint: You can use `len()` and `range()` functions.
- Append the name "Dastan" to the list. Print the list, did it change?
- Select the second player from the list and print: "[players_name] is the best player!" to the screen.
- Remove player "Abat" from the list.

Example output for the second subtask:

```
Players in reverse order:
Vyacheslav
Ramazan
Oralkhan
Abat
Maksim
```

5.4 Selling fruit (Spring 2025)

A salesperson sells fruit on the market. Create a list with the following pieces of fruit that he has initially:

[“banana”, “apple”, “kiwi”, “banana”, “banana”, “orange”, “apple”, “kiwi”, “strawberry”, “peach”]

Create an empty list/basket for a customer. Repeatedly prompt the user/customer what he likes to buy. If still available, remove once from the list and add to the basket of the customer. If not in the list (or not an existing word) print that you don’t have that. Stop when the user says: “That’s all”, and print the basket-list of the customer.

Possible output of a working script:

```
What do you like to buy: cherry
Sorry, today I don't have any cherry.
What do you like to buy: banana
Here you go.
What do you like to buy: apple
Here you go.
What do you like to buy: That's all
You bought following fruit:
["banana", "apple"]
```

6 Turtle graphics

6.1 Fibonacci square tiling with turtle

Turtle in a educational drawing tool built into Python. For more information about the turtle module, see the turtle documentation.

Generate Fibonacci numbers, see the problem of previous section, and use the turtle module to draw a Fibonacci tiling, see below image. To do this, you can use the Fibonacci numbers calculated in the previous task as the side lengths of the squares. Start with a square of side length 1, then draw a square of side length 1 next to it, then a square of side length 2 next to those two, and so on. Use the `forward()` and `right()` methods of the turtle to move it around and draw the squares. To draw a square of side length `side_length`, repeat the following steps four times:

```
my_turtle.forward(side_length)
my_turtle.left(90)
```

Hint: For drawing the next square, you want to go to the opposite corner and turn. Test out a couple of options till you get the correct pattern.

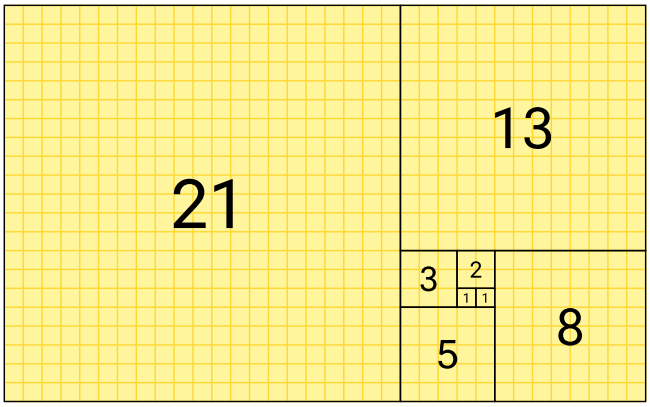


Figure 6.1: Fibonacci tiling: Spiral tiling with squares whose side lengths are successive Fibonacci numbers. From Wikipedia: By Romain - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=114415220

After you have made this pattern, try to use AI-assisted code generation to do the same exercise and compare your manually written code with the AI-generated code.

6.2 Fibonacci sunflower pattern with turtle

Make the turtle draw a sunflower pattern related to the Fibonacci sequence. The pattern is created by placing points at angles that are multiples of the golden angle $\phi = 180(3 - \sqrt{5}) \approx 137.5^\circ$ and at an increasing radius:

$$r = c\sqrt{n}, \qquad \theta = \phi n$$

To plot the points, first use `penup()` so you don't draw lines between the points, then use the `goto(x, y)` method of the turtle, where `x` and `y` are the Cartesian coordinates of the point. To plot the points use `dot(10)` where 10 is the size of the dot. Change the color by e.g. `color("red")`. You can convert from polar coordinates to Cartesian coordinates using the following formulas:

$$x = r\cos(\theta), \qquad y = r\sin(\theta)$$

See the below image for example output. Try to add more colors to the list of colors to change the appearance of the pattern.

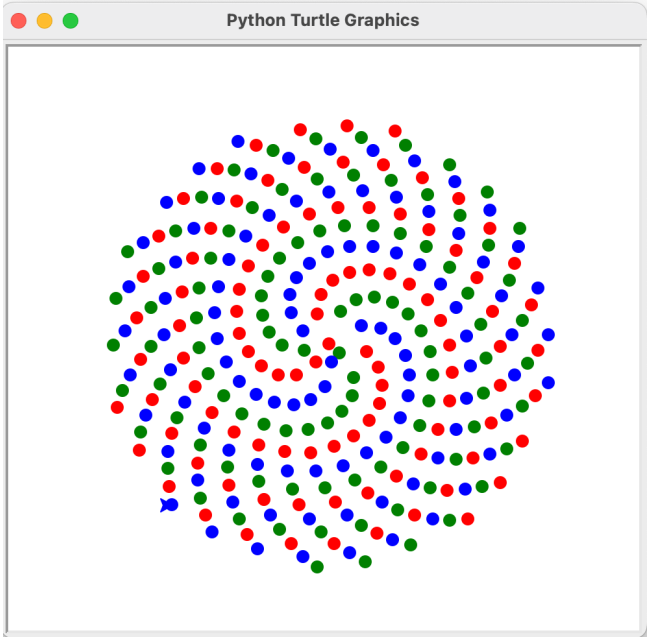


Figure 6.2: Sunflower pattern

6.3 Draw random dots with turtle

Plot N random dots on the screen using the turtle module. The position the dots should be randomly generated (Gaussian distribution), and the color and size of each dot should also be randomly generated. You can use the `random` module to generate random numbers for the position, color, and size of the dots. For example, you can use `random.gauss(mu, sigma)` for a Gaussian distribution with mean `mu` and standard deviation `sigma`, `random.uniform(a, b)` to generate a random float between `a` and `b`, and `random.choice(list)` to randomly select an element from a list. Output could look something like the below image.

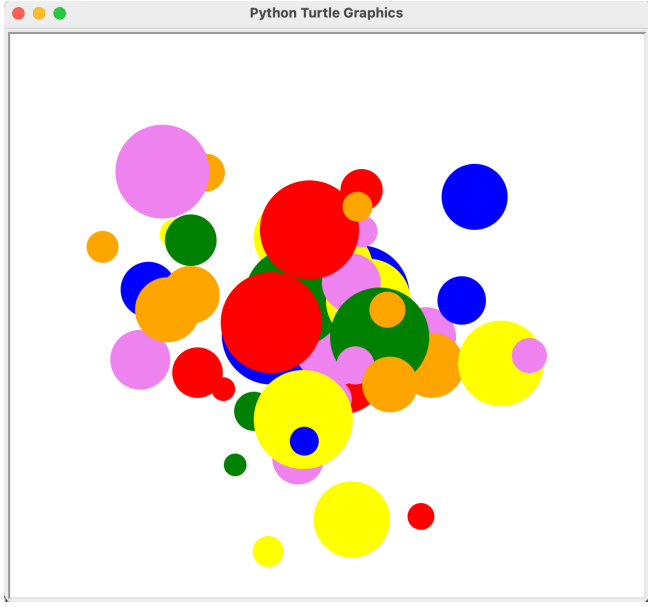


Figure 6.3: Random dots

7 Sequences and operations: Lists, Tuples, Strings

7.1 Making a tuple (Spring 2025)

Tuples are similar to lists but are immutable, i.e. they cannot be changed. You can create a tuple (list) using round (square) brackets.

- Make a tuple containing the odd elements: `1, 3, 5`
- Make a list containing even elements: `2, 4, 6`
- Use the `extend()` method on the list to add the elements of the tuple to the list. Print the extended list. There is no extend method for tuples, why not?
- Order the elements in the list from small to large using the `sort()` method on the list. Print the ordered list. Example output:

```
Extended list: [2, 4, 6, 1, 3, 5]
After sorting the elements of the list: [1, 2, 3, 4, 5, 6]
```

7.2 String chemistry (Spring 2025)

Take a long word such as "Dimethylaminophenyltrimethylpyrazoline" and apply the following string operations (print the result every time):

- Make all letters capital
- Inverse the order of the letters in the word
- Split the word into a list of strings whenever you encounter the letter "y" or "Y".
- Sort the letters alphabetically within each string of the splitted word.
- Print the second string of the list to the screen
- Create a string: "Chemistry is hard", add three exclamation marks "!" to the end, and repeat it 20 times (use the multiplication operator). Example output:

```
DIMETHYLAMINOPHENYLTRIMETHYLPYRAZOLINE
['DIMETH', 'LAMINOPHEN', 'LTRIMETH', 'LP', 'RAZOLINE']
['DEHIMT', 'AEHILMNNOP', 'EHILMRTT', 'LP', 'AEILNORZ']
AEHILMNNOP
chemistry is hard!!!chemistry is hard!!!chemistry is hard!!!chemistry is hard!!!chemistry is h
```

7.3 Yoda-talk (Spring 2025)

Prompt the user to give a sentence. Change the order of the words and print the new sentence. You can use following technique:

- Split the sentence in words.
- Mix up the order of the words (list of strings) by flipping every two word
- Use the `join()` method on the list of words to concatenate all strings in the list, so you obtain a single (mixed up) sentence again.

- Print this mixed up sentence to the screen.
- Try to mix up the order of the words more by using the shuffle function of the random module instead, see the example below:

```
import random

words = ["The", "force", "is", "strong", "in", "him"]
random.shuffle(words)
print(words)
```

```
['The', 'in', 'is', 'strong', 'force', 'him']
```

Example output:

```
Enter a sentence: The force is strong in him
is strong force him The in
```

8 Arrays

8.1 Create Numpy arrays (Spring 2025)

Unlike lists, *arrays* have a fixed length of elements, and all elements must have the same type (the elements of the array can be changed). We will use the Numpy package for arrays, **Numpy arrays are much faster** to access than lists.

You can create Numpy arrays in various ways:

```
import numpy as np

a1 = np.array([4, 2, 1])  # a1 = array created from a "2D" list
print(f"a1 = {a1}")
a2 = np.zeros((3, 1))     # a2 = 3x1 array (row-vector) with zeros
print(f"a2 = {a2}")
a3 = np.ones((4, 2))      # a3 = 4x2 array filled with ones
print(f"a3 = {a3}")
a4 = np.eye(2)            # a4 = 2x2 Identity matrix
print(f"a4 = {a4}")
```

Perform the following tasks:

- Convert the list: `[1, 2, 3]` to an array using the `np.array()` method.
- Create an 3×2 array filled with zeros using `np.zeros(n, m)`, print the array.
- Create an 1×5 array filled with ones using `np.ones(n, m)`, print the array.

8.2 Access elements in Numpy arrays (Spring 2025)

Accessing elements of arrays is done in the same manner.

```
# Get the 1th element of the array
i = 0
element = a1[i]
print(f"The element with index {i} = {element}")
```

```
The element with index 0 = 4
```

Perform the following tasks:

- Create a 3×3 array and fill the last column with numbers 1, 2, 3
- Extract the first row using the colon operator similar to lists
- Print the diagonal numbers using a loop

8.3 Different ways to generate the x-values: with Numpy (Spring 2025)

Numpy is designed for numerical computations so it has built-in functions for the generation of coordinates within intervals (see also https://numpy.org/doc/stable/user/how-to-partition.html#how-to-partition). Numpy functions `arange()` and `linspace()` create regularly spaced coordinates within 1D intervals:

```
import numpy as np
# Numpy: suppress scientific notation for small numbers and use 3 digits
np.set_printoptions(precision=3, suppress=True)

N = 10; A = -1; B = 2; dx = 0.25
x1 = np.arange(A, B, dx)   # x1 = points in interval [A, B[ with step dx
print(f"x1 = {x1}")
x2 = np.linspace(A, B, N+1)  # x2 = N points in interval [A, B]
print(f"x2 = {x2}")
```

```
x1 = [-1.   -0.75 -0.5  -0.25  0.    0.25  0.5   0.75  1.    1.25  1.5   1.75]
x2 = [-1.  -0.7 -0.4 -0.1  0.2  0.5  0.8  1.1  1.4  1.7  2. ]
```

The created coordinates are in Numpy arrays (not lists). Remember that **Numpy arrays are much faster** to access than lists.

- Create an array containing: `[1, 2, 3, 4, 5]`
- Create an array using `np.linspace(start, stop, step)` with 20 elements equally spaced in the interval $[-2, 1]$
- Create an array using `np.arange(start, stop, step)` with inter-element distance of 0.5 for the same interval $[-2, 1]$

8.4 Element-wise operations with Numpy arrays (Spring 2025)

Numpy also provides element-wise operations on Numpy arrays. We can use this to generate for example y-coordinates:

```
# Numpy arrays provide element-wise operations
y = x1**2  #  y contains squared array elements
print(f"y = {y}")
z = 5 * x1 + 3  #  z contains elements of x multiplied by 5 and increased with 3
print(f"z = {z}")
```

```
y = [1.    0.562 0.25  0.062 0.    0.062 0.25  0.562 1.    1.562 2.25  3.062]
z = [-2.   -0.75  0.5   1.75  3.    4.25  5.5   6.75  8.    9.25 10.5  11.75]
```

Numpy also provides functions on Numpy arrays:

```
res = np.exp(a1)
print(res)
a_degrees = np.array([45, 60, 30, 180, 90])
rads = np.deg2rad(a_degrees)
y = np.cos(rads)
print(y)
```

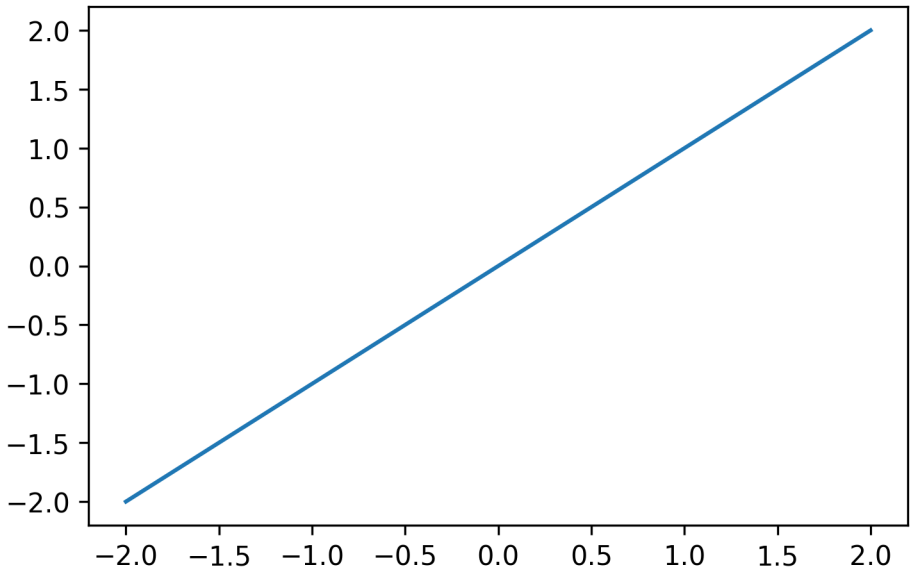
```
[54.598  7.389  2.718]
[ 0.707  0.5    0.866 -1.     0.   ]
```

- Create x and y coordinates for the expressions (take as interval $[-2, 2]$):

$$
\begin{aligned}
y &= x^3 \\
y &= \sin(\pi x) \\
y &= \sqrt{(}x + 2) \\
y &= \frac{\sin(\pi x)}{x}
\end{aligned}
$$

8.5 Expressions with Numpy arrays

Import the Numpy library and/or functions and take x now an array in interval $[-2, 2]$.



Plot the following formulas:

$$
\begin{aligned}
y &= x^2 \\
y &= \frac{\sin(\pi\, x)}{x + 5\pi} + 1 \\
y &= x^3 - 4x^2 + 2 \\
y &= \Re\{\sqrt{x}\}
\end{aligned}
$$

9 Plotting graphs with Matplotlib

Plotting a function is often done by approximating the function by piece-wise straight lines. For this we divide the interval that we want to plot in small pieces and generate (x, y)-coordinates. Suppose we want to plot the function $y = x^2$ over interval $x \in [A, B]$, then first we would create the x-coordinates:

$$x_i = A + i \cdot \frac{B - A}{N} = A + i \cdot \delta x \quad \text{with } i \in \{0, 1, .., N\}$$

with $\delta x = (B - A)/N$ the distance between the x-coordinates.

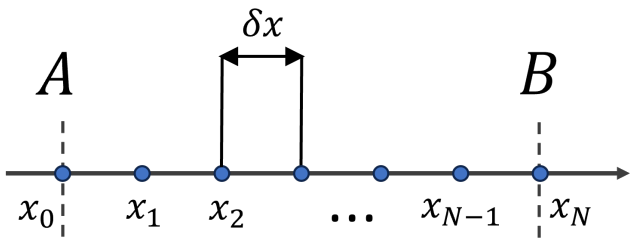
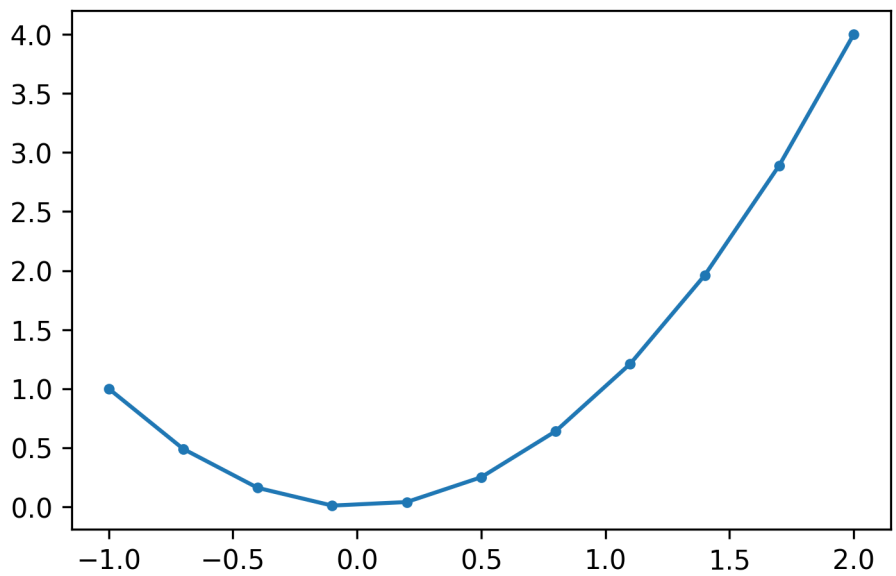


Figure 9.1: Interval divided in N pieces, i.e. a regularly spaced interval.

Afterwards we calculate the y-coordinates from the x-coordinates: $y_i = x_i^2$. Now that we have both coordinates we can plot the points with a line plot, which will connect the points with line pieces. The following code plots the curve $y = x^2$ within interval $[-1, 2]$



Within this example we used a low number of points, $N = 10$, which is too small to obtain a smooth realistic curve. Normally we plot curves with $N = 100$ or larger. I added dots on the curve (provided by the option `marker='.'` in the plot command) to indicate where the points are.

Different ways to generate the x-values

To create the $N+1$ coordinates x_i in interval $[A, B]$ there are various methods available. First I will list the methods not using the Numpy package:

(1) Using a `while` loop (not optimal when you know the number of divisions):

(2) Using a `for` loop and a `range`:

(3) Using list comprehension:

Different ways to generate the x-values: with Numpy

Numpy is designed for numerical computations so it has built-in functions for the generation of coordinates within intervals (see also https://numpy.org/doc/stable/user/how-to-partition.html#how-to-partition). Numpy functions `arange()` and `linspace()` create regularly spaced coordinates within 1D intervals:

```
x1 = [-1.   -0.75 -0.5  -0.25  0.    0.25  0.5   0.75  1.    1.25  1.5   1.75]
x2 = [-1.  -0.7 -0.4 -0.1  0.2  0.5  0.8  1.1  1.4  1.7  2. ]
```

The created coordinates are in Numpy arrays (not lists). **Numpy arrays are much faster** to access than lists. But lists can change their size easily and contain mixed object types.

Numpy also provides operations and functions on Numpy arrays. We can use this to generate for example the y-coordinates:

```
y = [1.    0.562 0.25  0.062 0.    0.062 0.25  0.562 1.    1.562 2.25  3.062]
z = [-2.   -0.75  0.5   1.75  3.    4.25  5.5   6.75  8.    9.25 10.5  11.75]
```

When to use which method ?

It seems that Numpy is much easier for creating **regularly spaced points within an interval**, especially when using the values afterwards within a calculation, so why would we use anything else? There are some details to it though, I tried to summarize the PRO's and CON's in the following table:

type	PRO's	CON's	Use-case
while loop	versatile, only option when number of iterations is unknown	cumbersome	Unknown number of iterations
for loop	Lower memory footprint if used with `range()`, lazy evaluation	Cumbersome	large number of iterations, performing a direct action each iteration
list comprehension	Clear, easy	Less convenient than Numpy `arange()`	When using Numpy is not desired/possible
Numpy arange	Easiest method for fixed step, Numpy provides array operations and functions	End of interval is exclusive, no lazy evaluation, slower than Python built-in `Array`	Regularly spaced intervals (fixed step) for computational purposes
Numpy linspace	Easiest method for fixed number of points, Numpy provides array operations and functions	No lazy evaluation, slower than Python built-in `Array`	Regularly spaced interval (fixed number of points) for computational purposes

Basically, if you need to create a regularly spaced interval:

- Use a `while` loop when you don't know the number of iterations up front.
- Use a `for` loop for huge number of iterations which cannot fit into memory, so that the lazy evaluation of `range` can help you there. In this case you would not create the actual list, but take an action at each iteration (e.g. save the result to a file).
- Use list comprehension if you don't want to use Numpy (or it is not available)
- Use Numpy's `arange` or `linspace` for numerical computations and plotting.

Note that this comparison is specifically for regularly spaced intervals. When iterating over lists of objects, using a `for` loop or list comprehension is more appropriate.

Before you start:

At the beginning of the script you should first import the required libraries (numpy and matplotlib):

```
# Import numpy and matplotlib
import numpy as np
import matplotlib.pyplot as plt
```

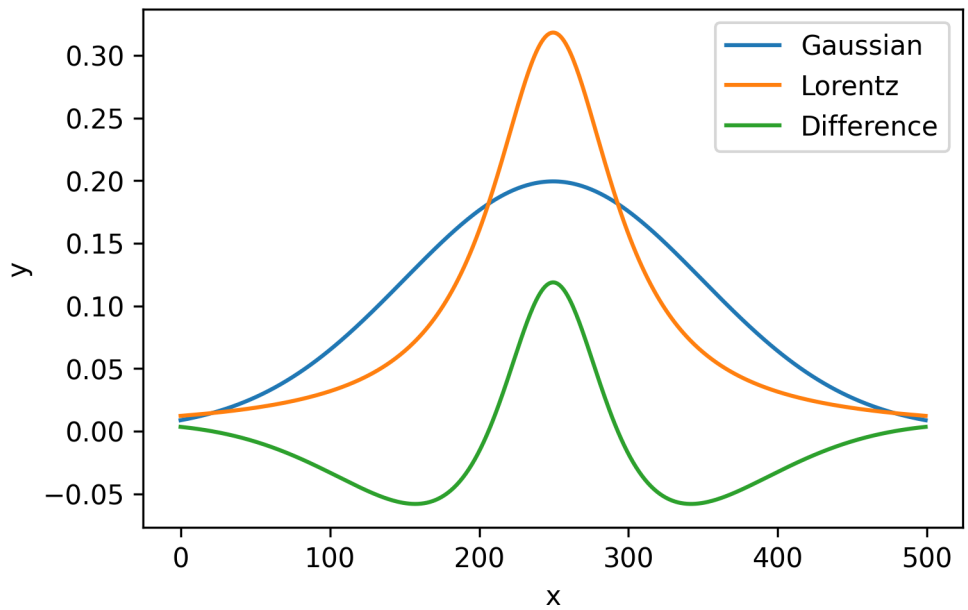
9.1 Plot a Gaussian and Lorentz distribution (Spring 2025)

Plot the Gaussian and the Lorentz function on one plot to compare them, add a legend and compute the curve of the difference between the two. The Gaussian curve is defined by:

$$g(x;\mu,\sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(x-\mu)^2/2\sigma^2}$$

The Lorentz curve is given by:

$$f(x;x_0,\gamma) = \frac{1}{\pi}\frac{\gamma}{(x-x_0)^2+\gamma^2}$$



9.2 Lissajoux figures (Spring 2024)

Lissajoux figures: sinusoidal signals in x and y.

$$x = \sin(f_x t + \delta\phi)$$
$$y = \sin(f_y t)$$

- Traversed coordinates in time `t` form shapes
- Signal frequencies `fx` and `fy`
- Phase difference `dphi`

Lissajoux figures are typically encountered on an oscilloscope (see the figure below). In an oscilloscope electric signals are connected to the vertical and horizontal

```
# Use following parameters
fx = 3; fy = fx - 1; dphi = pi/2 # or dphi = pi/4

# Hint: Use np.linspace(), np.pi, np.sin()
#       take time in interval [0, 2*pi], e.g.:
#           t = np.linspace(0, 2*np.pi, 1000)
```

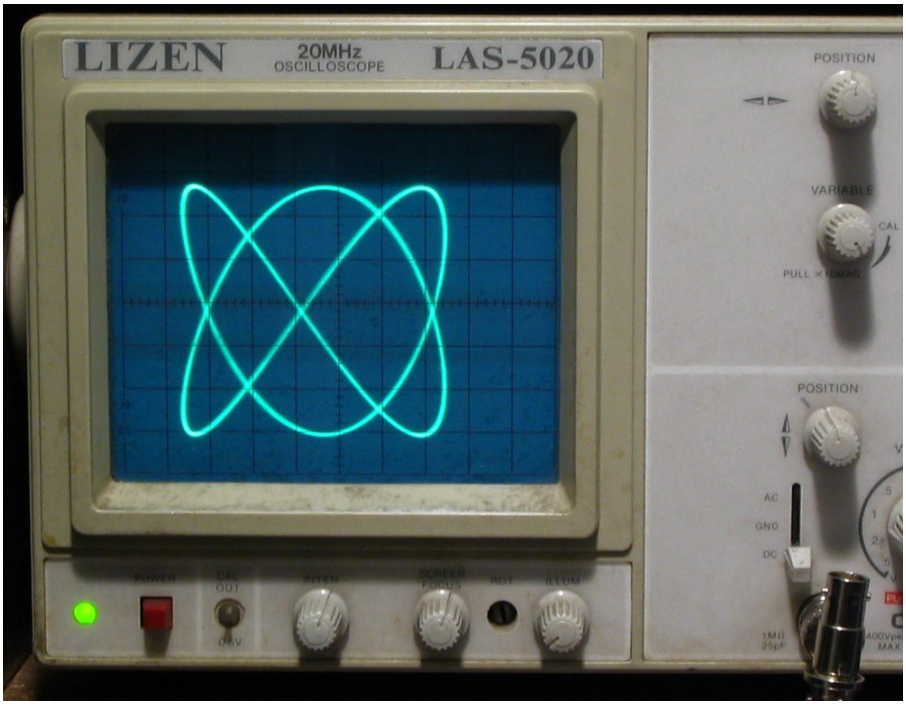
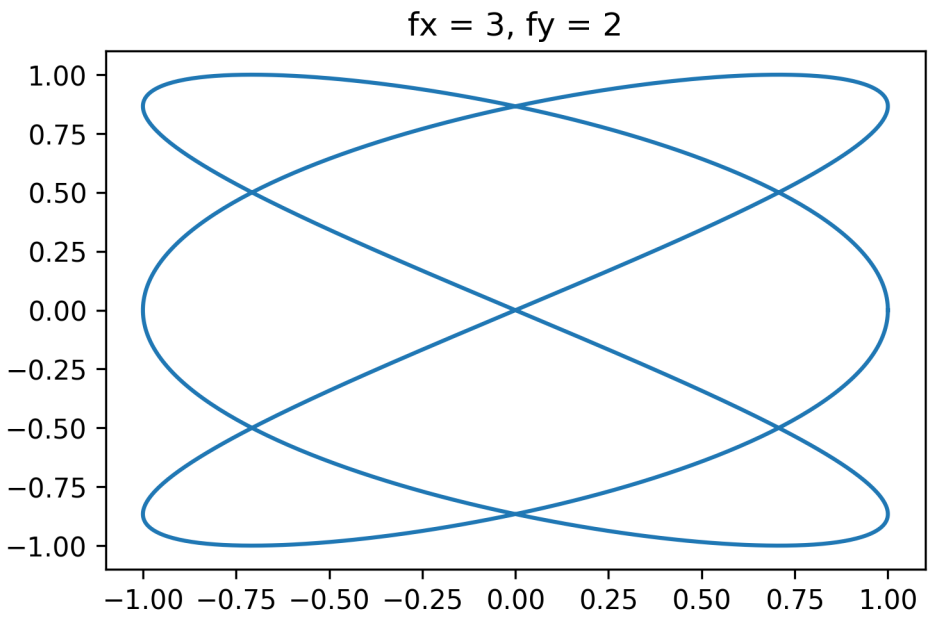


Figure 9.2: An oscilloscope allows to see the correlation between two electric signals, by plotting one signal horizontally and the second vertically.

Output plot example



9.3 Refraction at a glass interface (Spring 2024)

Plot the refracted angle θ_2 (using Snell's law) of a ray of light incident at a glass medium, as function of the incoming angle: use the incoming angle $\theta_1 \in [-90, 90]$ degrees medium 1 has $n_1 = 1$, medium 2 has $n_2 = 1.55$.

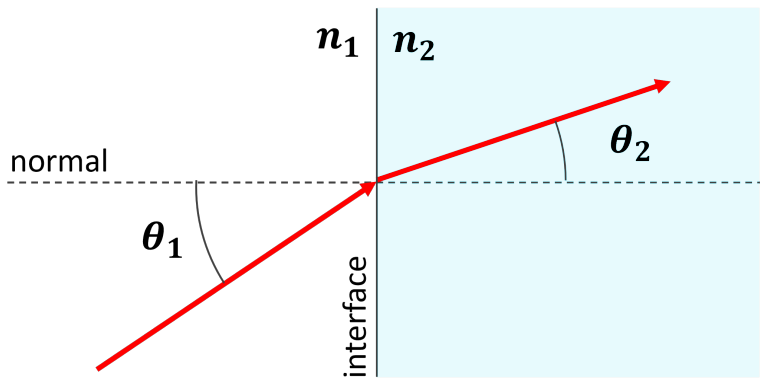


Figure 9.3: Snell's law

Snell's law is given by:

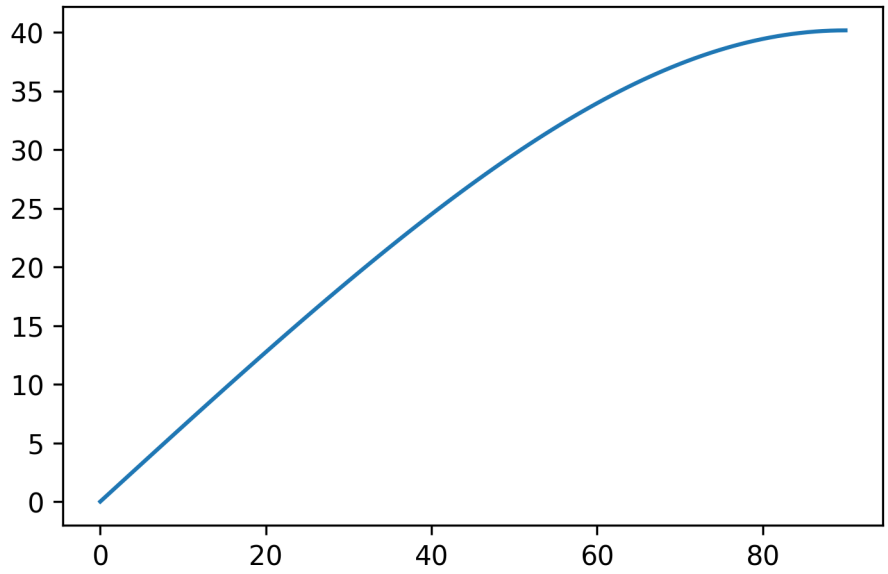
$$n_1 \sin(\theta_1) = n_2 \sin(\theta_2)$$

If refraction indices n_1 and n_2 are known, the refracted angle can be calculated as follows:

$$\theta_2 = \arcsin\left(\frac{n_2}{n_1}\sin(\theta_2)\right)$$

Hint: Use Numpy arrays for the angles (np.linspace()), and Numpy functions: np.sin(), np.arcsin() (convert to radians by multiplying by a factor pi/180) or use the functions np.rad2deg and np.deg2rad to convert between the two.

The output of your code should be similar to following plot:



9.4 Adapt previous plot to include axis labels, etc. (Spring 2024)

- Add labels to your axes using the `set_xlabel` and `set_ylabel` methods. Put $-symbols around mathematical/Greek letters and double-escaped Greek letters such as theta (the escape symbol is "\"). For example:

```
ax.set_xlabel("$\\theta_1$ (in degrees)")
```

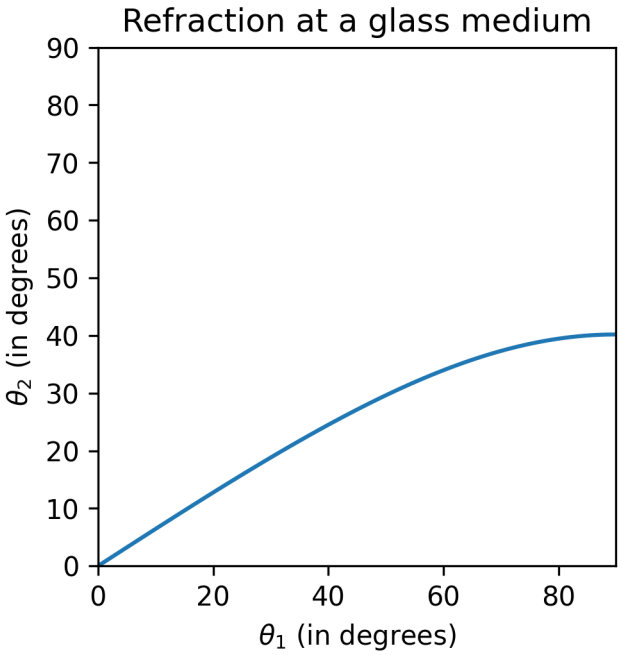
- Add a title to your plot (e.g. "Refraction angles") by using the method: `ax.set_title()`
- Set the intervals to be plotted between 0 and 90 degrees, and set the aspect ratio of the axes equal to one:

```
    ax.set_xlim([0, 90])
    ax.set_ylim([0, 90])
    ax.set_aspect('equal', 'box')
```

- Save the figure using fig.savefig() as a png figure, for example:

```
    fig.savefig("./ex2_figure.png")
```

The output should give you a similar plot to the one below:



9.5 Extend previous script to show multiple incident angles (Spring 2024)

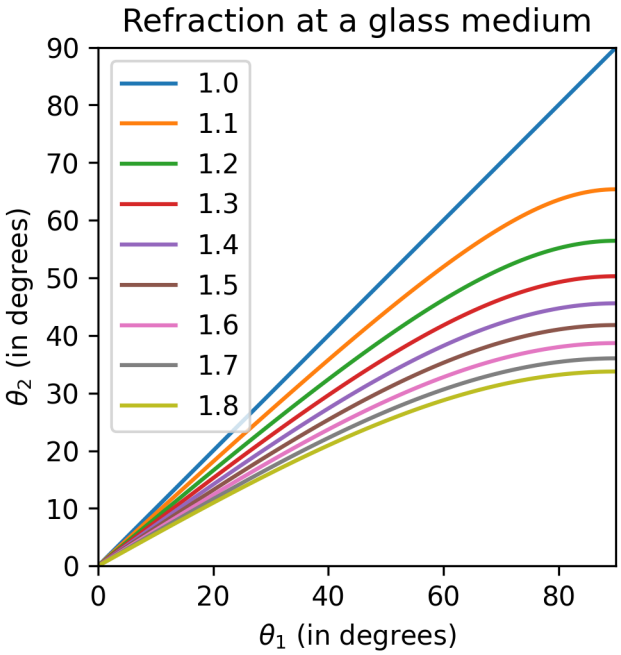
Plot the previous plot but now with multiple values for $n_2 \in [1, 1.8]$

- use the arange function of Numpy to create values for n_2 within the interval, for example each value separated by $\delta n = 0.1$

```
n2s = np.arange(1, 1.85, 0.1)
```

- Plot in the same axis when looping over the values (but initialize the `fig, ax` only once before the loop)
- add a legend using `plt.legend(n2s)`
- to remove the extra decimals due to rounding errors, use `plt.legend(np.round(n2s, decimals=2))` to round the numbers before showing the legend.

The output should give the below plot:



9.6 Plot ellipses with various parameters (Spring 2024)

Plot ellipses as parametric plots with parameter $t \in [0, 2 * \pi]$ and where the (x, y)-curve is given by:

$$x = a\cos(t)$$
$$y = b\sin(t)$$

Plot multiple ellipses with different values of a and b:

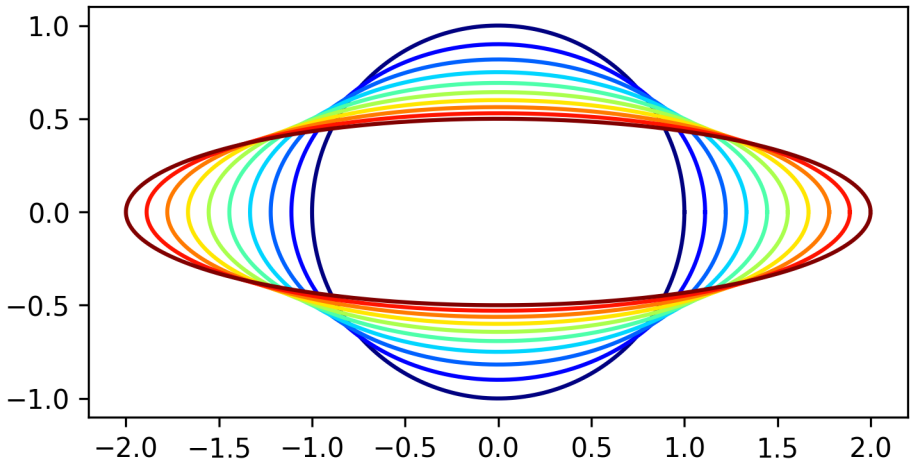
- take parameter $a \in [1, 2]$ and choose a small (e.g. 8) curves using `np.linspace()`, afterwards calculate corresponding value for parameter b from a by using $b = 1/a$
- give every ellipse a different color and choose the color palette Matplotlib uses by using e.g.:

```
colors = plt.cm.jet(np.linspace(0, 1, n_curves))

# Then afterwards define the line color:
ax.plot(xs, ys, color=colors[i])
```

- use: `ax.set_aspect('equal', 'box')` to set the aspect ratio of the x and y axis equal

The output of the script should give the following plot:



9.7 Plot Poinsot's spiral (2nd form) (Spring 2024)

Poinsot's spiral is defined in polar coordinates as:

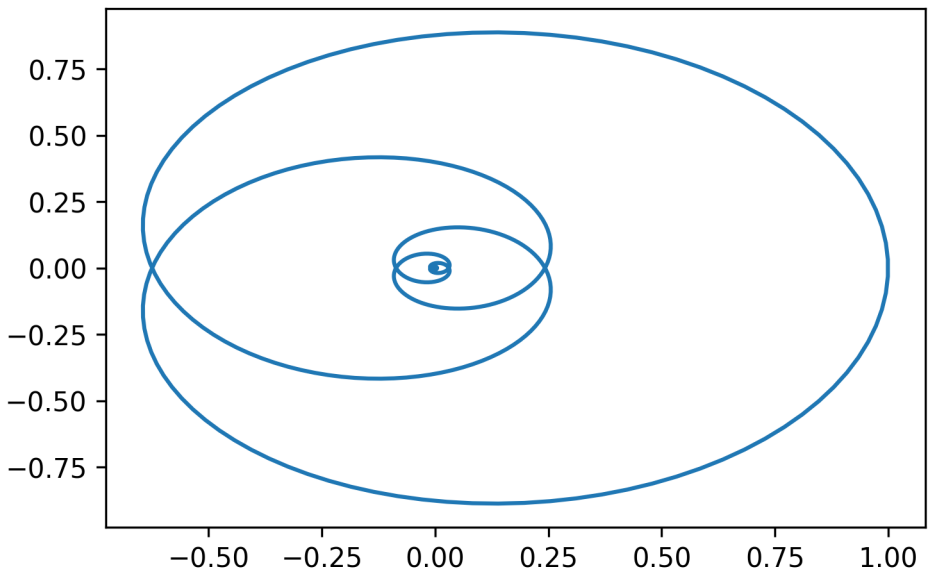
$$r = \frac{1}{\cosh(n\theta)}$$

plot with θ in interval $[-10 * pi, 10 * pi]$ and convert from polar coordinates to (x,y)-coordinates by

$$x = r\cos(t)$$
$$y = r\sin(t)$$

- Take the parameter n for example $n = 1/3$ to have a typical curve

The output plot should look as below:



9.8 Plot a Gaussian distribution (Spring 2024)

Generate 10000 random numbers with a normal distribution and plot the histogram - First initialize the random number generator

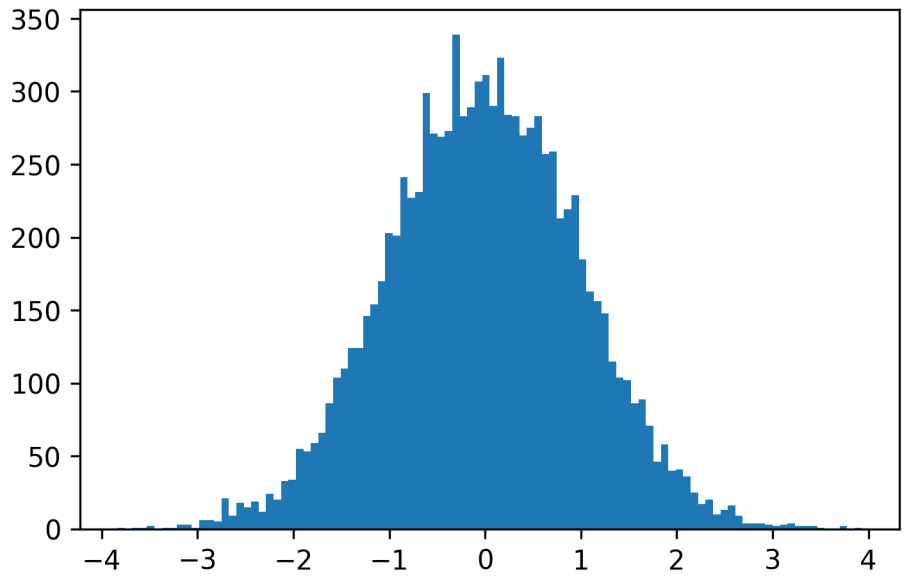
```
rng = np.random.default_rng(1)
```

- use the random number generator we created to generate 10000 random numbers from the standard normal distribution using:

```
xs = rng.standard_normal(size=10000)
```

- plot a histogram using `ax.hist(x_uniform, 100)`

The resulting plot should be:



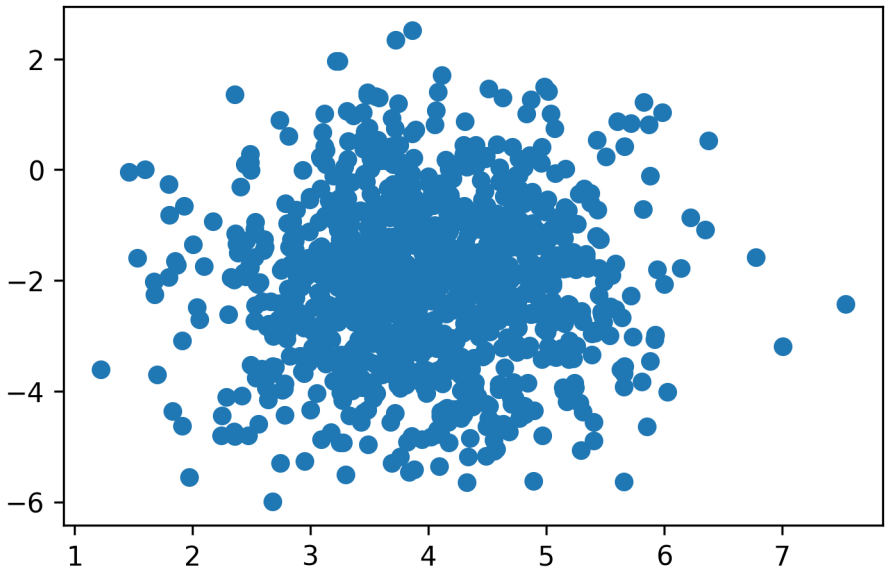
9.9 Make a scatter plot of a Gaussian distribution in 2D (Spring 2024)

- use the random number generator we created to generate 1000 random numbers for both x and y coordinates,
- Use the normal distribution
- fill in `loc(=mean)`, and `scale(=sigma)` as extra parameters:

```
xs = rng.normal(loc=4, scale=0.9, size=1000)
ys = rng.normal(loc=-2, scale=1.5, size=1000)
```

- plot them in a scatter plot using: `ax.scatter(xs, ys)`

The resulting output should look as below:



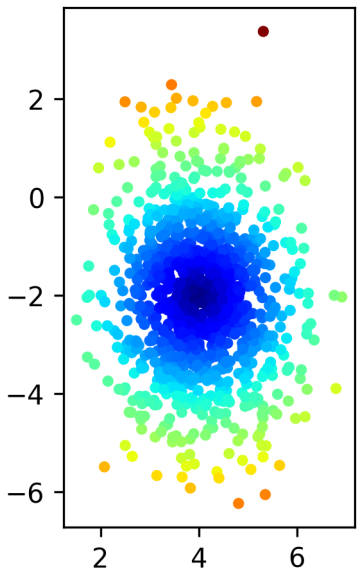
9.10 Adapt the scatter plot of previous exercise by specifying the size and color of the dots (Spring 2024)

```
ax.scatter(xs, ys, sz, colors, cmap=matplotlib.colormaps["jet"])
```

where both `sz` and `colors` can be arrays of the same size as the coordinates or scalars

- for the size use for example 10
- for the colors create an array containing the distance of the coordinates to the center of the distribution
- the cmap argument allows us to set the colormap used

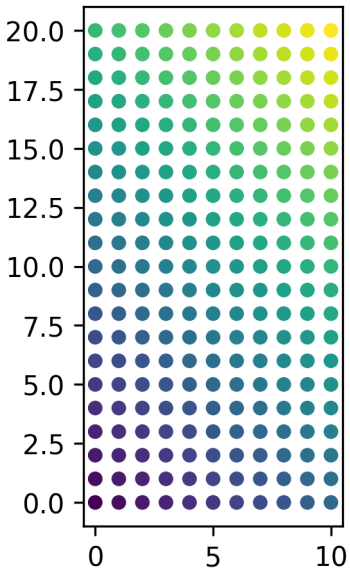
The resulting scatter plot should look as below:



9.11 2D domains (Spring 2024)

Use Numpy's `arange()` function to create two intervals containing integers with e.g. $x \in [0, 10]$ and $y \in [0, 20]$, and make a 2D domain of them using the `xx, yy = np.meshgrid(x, y)` function. Plot the coordinates thus created using a scatter plot (`plt.scatter(xx, yy)`).

The plot should similar to the following:



9.12 2D arrays (Spring 2024)

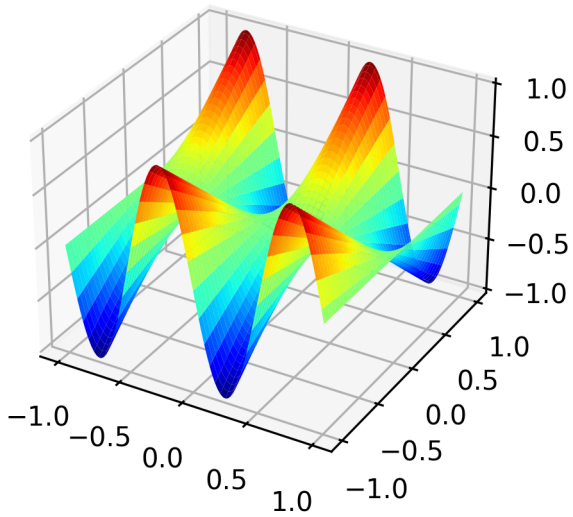
Use a 2D domain/interval using the `np.meshgrid` command such as in exercise 1, but using `np.linspace()` for the intervals with $x \in [-1, 1]$ and $y \in [-1, 1]$ at a finer grid, and plot the function for that domain:

$$z = f(x, y) = y \sin(2\pi\, x)$$

For the plot use the `plot_surface()` from Matplotlib:

```
fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
ax.plot_surface(xx, yy, zz, cmap=cm.jet)
plt.show()
```

The output should be similar as below:


1.0
0.5
0.0
−0.5
−1.0
1.0
0.5
0.0
−0.5
−1.0
−1.0
−0.5
0.0
0.5
1.0

10 Exercises on functions

We will exercise creating functions and use them within problems similar to the ones we encountered before during recitation. Remember the different elements of the function definition:

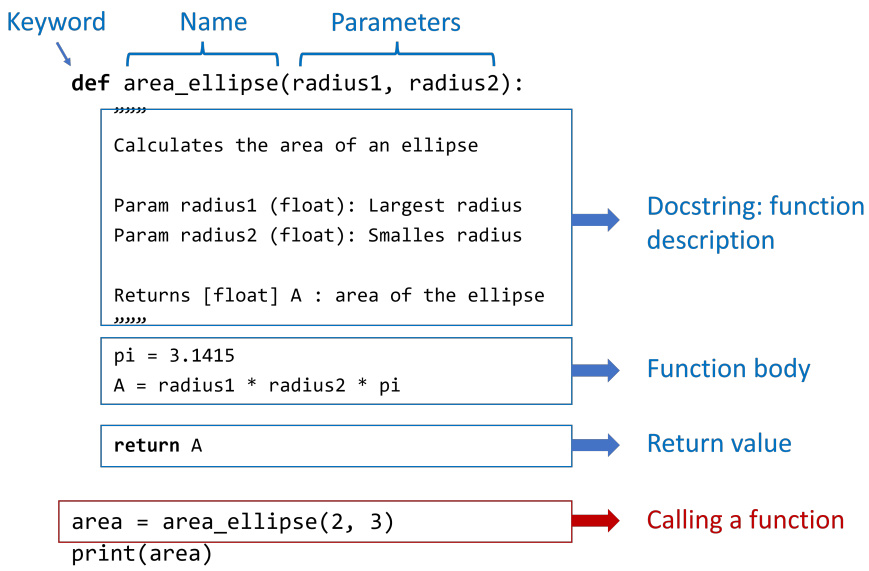


Figure 10.1: Function definition elements

At the beginning of the script you should first import the required libraries (numpy and matplotlib):

```
# Import numpy and matplotlib
import numpy as np
import matplotlib.pyplot as plt
```

10.1 Calculate the factorial of a number (Spring 2024)

Calculate the value of the factorial of a number by defining a new function: `factorial(n)`. Use the following script as basis:

```
The value of 5! = 120
```

10.2 Calculate the number of combinations (Spring 2024)

The number of combinations of k items out of a set of n objects is defined in statistics as

$$\mathcal{C}_k^n = \binom{n}{k} = \frac{n!}{(n-k)!\,k!}$$

where $k \leq n$.

- Use the function `factorial(n)` which you created in previous exercise 1 to calculate the number of combinations $\mathcal{C}_3^5$.
- Afterwards create a function `combinations(n, k)` to compute the combinations.

10.3 Intersection of two lines (Spring 2024)

Find the intersection point $p = (x_p, y_p)$ between two lines with equations

$$\begin{cases} y = m_1\, x + c_1 \\ y = m_2\, x + c_2 \end{cases}$$

To find the intersection we extract the x-value by making use of the fact that at the intersection the y-values should be equal.

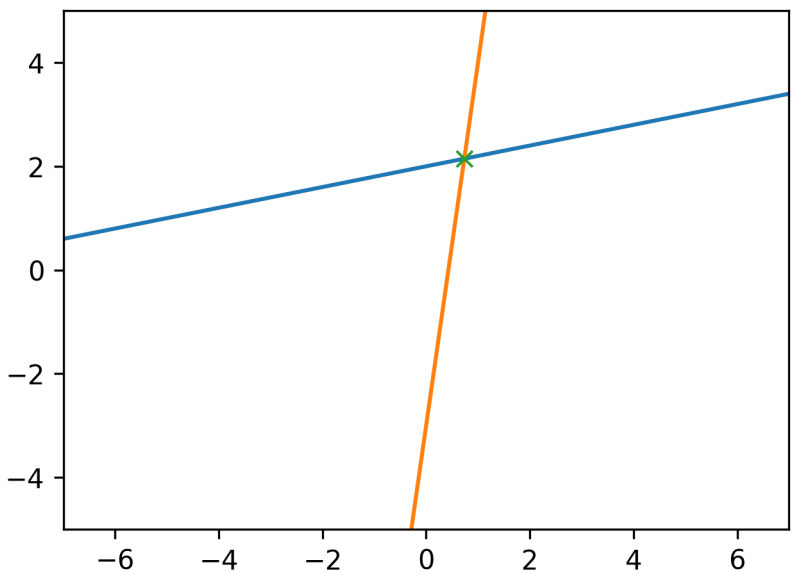
$$\begin{aligned} m_2\, x_p + c_2 &= m_1\, x_p + c_1 \\ \Rightarrow (m_2 - m_1)\, x_p &= c_1 - c_2 \\ \Rightarrow x_p &= -\frac{c_2 - c_1}{m_2 - m_1} \end{aligned}$$

then we substitute the found x_p coordinate into one of the equations of the system to obtain the y_p coordinate:

$$y_p = m_1\, x_p + c_1$$

As example parameters of the lines: pick $m_1 = 1/5$ and $m_2 = 7$ as the direction coefficients, and $c_1 = 2$ and $c_2 = -3$ the off-sets at $x = 0$.

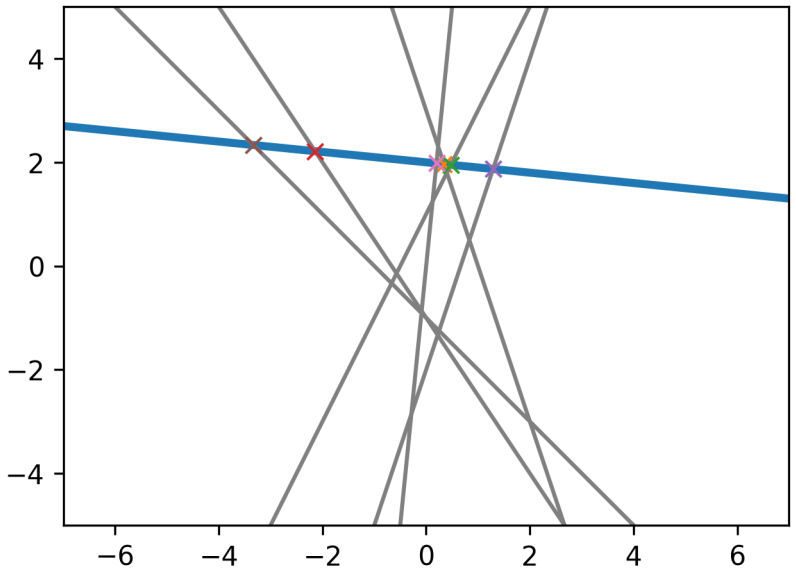
Convert the code to calculate the intersection point in following script into a function: `calc_intersection(m1, c1, m2, c2)` which returns `xp, yp`. Afterwards use your new function within this script.



10.4 Intersection of many lines (Spring 2024)

Use the function that you created in previous exercise to calculate the intersection of a line with a list of other lines defined by:

The output plot should look as follows:



10.5 Drawing the Koch curve (Spring 2025)

Fractals are easiest defined by recursive algorithms. Look at the following code which draws (one side of) the Koch curve.

- Afterwards try to adapt the code to plot the whole Koch curve (the three sides of the snowflake)

```
import turtle

def koch(a, order):
    if order > 0:
        for t in [60, -120, 60, 0]:
            koch(a/3, order-1)
            turtle.left(t)
    else:
        turtle.forward(a)

# Draw the Koch curve using Turtle
turtle.setworldcoordinates(-10, -10, 100, 100)
turtle.pensize(3)
turtle.pencolor("blue")
turtle.speed(10) # possible issue
koch(100, 4)
```

11 Exercises on error handling

11.1 User console input (Spring 2024)

When we ask a user to fill in a form or type something at the prompt, we assume a certain format. Let's take the example where we ask how tall a person is. Hereby we expect to get the answer as an integer number in centimeters. However, when prompted with this question, a user can fill in something invalid (e.g. "1m72"). When that happens we want to allow the user to try again and maybe give an extra hint.

A typical manner to arrange this is:

```
while True:
  try:
    height = int(input("\tYour height (in cm) = "))
    print(f"You are {height} cm")
  except ValueError:
    print(f"Invalid input: please try again")
```

- Adapt the above code so that it accepts decimal input such as "174.7"

11.2 Read from a text file (Spring 2024)

- Adapt the code to read a text file on your computer of exercise one so that it doesn't crash when the file path doesn't exist.
- Prompt the user to choose the file name.

12 Python Custom Modules

Modules are separate python script files which contain functions and variables to be imported in other scripts.

12.1 A simple module for calculations (Spring 2025)

Add the following functions to a separate module script called `my_module.py`

```
def add(a, b):
    """ adds a + b """
    return a + b


def mul(a, b):
    """ multiplies a * b """
    return a * b
```

Then import this module into your main script-file and call the module functions to perform the following calculations: $x = 6 + 43$ and $y = (3 + 6) * 4$ and print the results to the console.

12.2 A module for light at an interface (Spring 2025)

Create a module to calculate the reflected light rays and refracted light rays at a vertical interface. You can use the following example code:

Listing 12.1 `themodule.py`

```
def calc_reflection_angle(angle_inc):
  # ...
  # code to calculate the reflection at an interface
  # ...
  return angle_reflection

def calc_refraction_angle(angle_inc, n1, n2):
  # ...
  # code to calculate the refraction at an interface
  # ...
  return angle_refraction
```

Listing 12.2 `thescript.py`

```
# Import your module
import themodule as lm

# Parameters
angle_inc = 45

# Calculate the angle of reflection at a vertical interface
angle_reflection = lm.calc_reflection_angle(angle_inc)
print(angle_reflection)

# Calculate the angle of refraction at a vertical interface
# with media n1 = 1, and n2 = 1.5
angle_refraction = lm.calc_refraction_angle(angle_inc, 1, 1.5)
print(angle_refraction)
```

12.3 A module for converting text (Spring 2025)

Create a module with a function to convert text into upper-case text with all exclamation marks (`!`) removed. Name the module file `my_io.py`. In your main script do the following:

- Import the module
- Ask the user for an input sentence
- Convert the sentence by using the function of the module

Then add another function to the module to convert text in a text file into a minimal html page. You can use the following example code to convert the text into html:

```
html_text = """<!doctype html>
<html lang=en>
    <head>
        <meta charset=utf-8>
        <title>""" + title + """</title>
    </head>
    <body>
        <p>""" + text + """</p>
    </body>
</html>
"""
```

Provide the main script with a sample text file and after conversion try to open your html-file.

13 Dictionaries & data tables (Pandas dataframes)

13.1 Creating a dictionary (Spring 2025)

You can create a dictionary using curly brackets

```
position = {"x": 1, "y": 1}
```

Perform the following tasks:

- Create a dictionary with four variables (position in space-time): "x": 0, "y": 1, "z": 0, "t": 0.
- Add this dictionary to an empty list.
- Then create a function that allows to add another dictionary of space-time coordinates
- Afterwards, create another function that allows you to convert the list of space-time coordinates to a single dictionary of 4 arrays x, y, z, and t, containing the coordinates.

13.2 Convert to a module (Spring 2025)

Put the functionality of previous exercise in a module. Remember to use a separate script-file for the module.

- Test each of the functions of the module.
- Afterwards, try to import the module in a main script which plots the space-time coordinates in a 3D parametric curve plot.

```
def add_position(positions, x, y, z, t):
  # ...
  # code to add extend the list
  # ...
  return positions

def convert_to_dict(positions):
  # ...
  # code to convert the list of
  # dictionaries into a dictionary of lists
  # ...
  return coords
```

13.3 More advanced datasets: Pandas package

13.4 Download a dataset

For this exercise you first need to install the python package `scikit-learn` (via `pip install scikit-learn`). Be aware that the module name inside the package is different, importing the package is via `import sklearn`. Then perform the following steps:

- Go to the Scikit-learn website:

```
ModuleNotFoundError: No module named 'sklearn'
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[2], line 1
----> 1 from sklearn.datasets import load_iris
      3 iris = load_iris()
      4 iris.head()
ModuleNotFoundError: No module named 'sklearn'
```

14 Classes

14.1 Library system

Use the code below to define a new class called `Book`.

```python
class Book():
    """Book class defines books in a library"""

    # Attributes of the class
    title = "Nasreddin Hoca Fıkraları"
    author = "Memet Fuat"
    category = "Philosophy"
    language = "TR"

    # Methods of the class
    def print_info(self):
        print(f"'{self.title}' by {self.author}")


if __name__ == "__main__":
    library = []
    book = Book()
    print("The first default book is:")
    book.print_info()
    library.append(book)
```

```
The first default book is:
'Nasreddin Hoca Fıkraları' by Memet Fuat
```

Afterwards, make the following adaptations/extensions:

- Add a constructor method to the `Book` class using `__init__(self, title, etc.)`, and your favorite book to the `library`.
- Add a for-loop to print the information of each book.
- Adapt the constructor to include two extra attributes:
  - The id of the book as a string, example: `id = "AB-14"`,
  - The total number of books with that id: `n_total = 1`,
  - The number of available books with that id: `n_available = 1`
- Add an extra method `borrow(book_id)` which:
  - Returns with a confirmation message, and reduces the number of available books by one if there are still books with that id.
  - Returns with a message: "The book: [the_book_title] is not available at the moment." if there are no books available anymore.

14.2 Subclasses

First define a class: `Ticket`

```
class Ticket():
    """The Ticket class is defined for general transport vehicles"""

    # Attributes of the class
    customer = "your_name"
    price = 20

    # Methods of the class
    # ...
```

- Add a constructor (`__init__(self, customer, price, etc.)`) for the class,
- Then add a method which
- Then create a derived class `Metro` (subclass), for which you add the following attributes:

  ```
  valid_from_time = [get_the_current_date_time]
  valid_until_time = [valid_from_time + 2 hours]
  ```

- Afterwards create an instance of your new Metro class and check whether the `valid_until_time` is correct.

For the date and time in Python you can use the built-in `datetime` package. See the example below:

```
from datetime import datetime, timedelta

current_time = datetime.now()
current_time_str = current_time.strftime("%Y-%m-%d %H:%M:%S")
print("It is now: " + current_time_str)

later_time = current_time + timedelta(hours=3)
later_time_str = later_time.strftime("%H:%M:%S")
print("Three hours later the time will be: " + later_time_str)
```

```
It is now: 2026-03-25 20:26:57
Three hours later the time will be: 23:26:57
```

15 Numerical algorithms

15.1 Compute an integral numerically (Spring 2024)

Numerically integrate the following integral

$$\int_0^2 \mathrm{d}x\,(x^3 - x/3)$$

(1) As a first approximation try to use the rectangular approximation (Riemann sum). Therefore, divide the interval $[0, 2]$ in N parts and approximate the area surrounding each point by a thin rectangle. The sum of all rectangle areas then approximates the total area under the curve (see the figure below), i.e. the integral. For a general interval $[A, B]$ and function $y = f(x)$ we can numerically calculate the integral using:

$$\int_A^B \mathrm{d}x\, f(x) \approx \sum_{i=1}^{N} f(x_i)\delta x$$

With $x_i = A + (i - 1/2) \times \delta x$, and $\delta x = (B - A)/N$. This sum is also called **Riemann sum**. In principle, we should take the limit $N \to \infty$ to obtain the analytic result or actual Riemann integral, but as an approximation we take N a finite (large number).

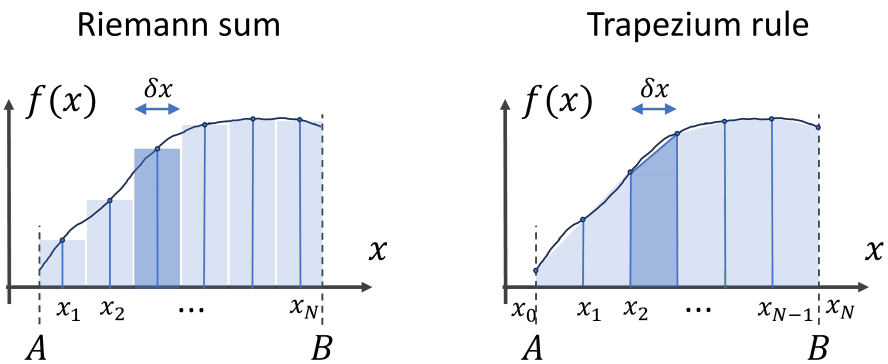


Figure 15.1: Numerical integration of a definite integral over interval $[A, B]$ interval. The **left** panel illustrates the Riemann sum, where the area under the curve is approximated by rectangles. On the **right** the Trapezium rule is illustrated where the area is approximated by trapeziums instead of rectangles, increasing the accuracy.

(2) Compare the result with the analytic result:

$$[x^4/4 - x^2/6\,]_0^2 = 10/3$$

(3) Then use the **trapezium rule**, see the figure above, in which instead of rectangles we use trapeziums, following the curve more accurately. The area of a trapezium with two vertical sides and one horizontal bottom side is given the width times the average length of the vertical sides.

$$\int_A^B \mathrm{d}x\, f(x) \approx \sum_{i=1}^{N} \frac{f(x_i) + f(x_{i-1})}{2} \delta x$$

With $x_i = A + i \times \delta x$, and $\delta x = (B - A)/N$. This sum should more accurately reflect the area under the curve (the integral) than the Riemann sum.

(4) This formula is used so often that Numpy has a function for it: `np.trapz(y, x)`. Compare your result with the result of this function.

15.2 Compute derivatives numerically (Spring 2024)

The derivative of a function in a point can be numerically approximated by finite difference methods. The following formula gives us the **forward finite difference**:

$$\frac{\mathrm{d}f(x)}{\mathrm{d}x} \approx \frac{f(x_{i+1}) - f(x_i)}{\delta x}$$

If we would take the limit of $\delta x \to 0$ then we would obtain the actual derivative. Here however we will approximate the derivative with a finite-sized δx. We we want to calculate the derivative over an interval we will as we did for the integral divide the interval in N pieces and set $\delta x = (B - A)/N$. Thereby we can plot the derivative over the whole interval $[A, B]$ (except of the last point at B, for which we can't compute the forward derivative). Similarly, the **backward finite difference** is defined as:

$$\frac{\mathrm{d}f(x)}{\mathrm{d}x} \approx \frac{f(x_i) - f(x_{i-1})}{\delta x}$$

And the mean of the two becomes the **central difference**:

$$\frac{\mathrm{d}f(x)}{\mathrm{d}x} \approx \frac{1}{2}\left(\frac{f(x_{i+1}) - f(x_i)}{\delta x} + \frac{f(x_i) - f(x_{i-1})}{\delta x}\right) = \frac{f(x_{i+1}) - f(x_{i-1})}{2\delta x}$$

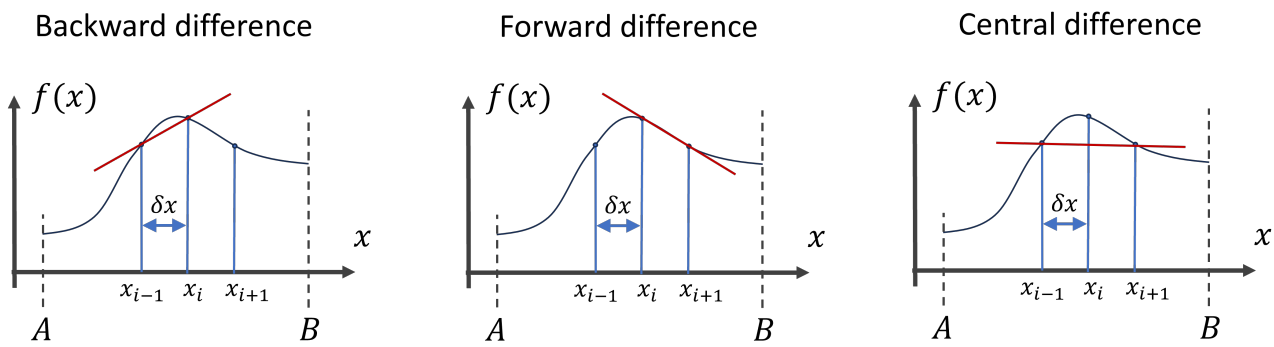


Figure 15.2: Numerical derivatives. The central difference allows the most accurate derivative and can be considered as the mean derivative of the forward and the backward difference approximation.

(1) Numerically compute and plot the forward derivative of $f(x) = \sin(5\pi x)/(1+x^2)$ within interval $[-3, 3]$. Ignore the issue at the last point.
(2) Do the same for the central difference, do you find any difference for small values of N?

15.3 Solve a system of equations

Solve the following system of equations in x, y, and z:

$$\begin{cases} x + y = 0 \\ x + y + z = 5 \\ 2x - z = -2 \end{cases}$$

by converting it to a matrix equation:

$$\begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 2 & 0 & -1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 0 \\ 5 \\ -2 \end{pmatrix}$$

And then multiplying both sides of the equation by the inverse matrix from the left.

$$\begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 2 & 0 & -1 \end{pmatrix}^{-1} \begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 2 & 0 & -1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 2 & 0 & -1 \end{pmatrix}^{-1} \begin{pmatrix} 0 \\ 5 \\ -2 \end{pmatrix}$$

$$\Rightarrow \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 1 \\ 2 & 0 & -1 \end{pmatrix}^{-1} \begin{pmatrix} 0 \\ 5 \\ -2 \end{pmatrix}$$

Thereby calculate and print the values for x, y, and z. Verify by hand whether this is indeed a solution.

Solutions

Input, formatting output, and expressions in Python scripts

2.1 Body Mass Index calculator [Spring, 2025-2026]

Listing 15.1 code/2026/ex_2_2.1.py

```
# Practical 2: Exercise 2.1
h = float(input("Please provide your height in centimeters: "))
w = float(input("Please provide your weigth in kilograms: "))
print(f"Your BMI = {w/(h/100)**2:4.4}")
```

2.2 Text transformations [Spring, 2025-2026]

Listing 15.2 2026/ex_2_2.2.py

```
# Practical 2: Exercise 2.2
s = input("Please provide a sentence: ")
print(f"(a) Uppercase sentence: {s.upper()}")
print(f"(b) With quotes: {'"' + s + '"'}")
n = 4
print(f"(c) Repeated {n} times: {n * (s + "?")}")
print(f"(d) On separate lines:\n{s.replace(" ", "\n")}")
```

2.3 Tsiolkovsky rocket equation [Spring, 2025-2026]

2.4 Formatted table: unit conversion [Spring, 2025-2026]

2.5 Glorifier [Spring, 2024-2025]

2.6 Repeat words [Spring, 2024-2025]

2.7 Braking distance [Spring, 2024-2025]

Listing 15.3 2026/ex_2_2.3.py

```python
# Practical 2: Exercise 2.3

# Import statements
from math import exp

# Parameters
dv = 7.8
ve = 2.5

# Input
ms = float(input("Mass of the satelite (kg): "))
mr = float(input("Mass of the empty rocket (kg): "))

# Calculate require mass of fuel
mf = ((ms + mr) - 1) * exp(dv / ve)

# Output
print(f"==> Required fuel = {mf:4.4} kg")
```

2.8 Velocity measurement by radar gun [Spring, 2024-2025]

2.9 Chevaux-Vapeur car tax calculator [Spring, 2024-2025]

2.10 Formatted table of numbers [Spring, 2024-2025]

Conditional Statements

3.1 Check login [Spring, 2025-2026]

3.2 Roots of a quadratic equation [Spring, 2025-2026]

For and While loops

4.1 Number series [Spring, 2025-2026]

4.2 Power series [Spring, 2025-2026]

4.3 Printing numbers [Spring, 2024-2025]

Listing 15.4 2026/ex_2_2.4.py

```python
# Practical 2: Exercise 2.4

# Parameters
inch = 0.01 * 2.54
foot = 12 * inch
yard = 3 * foot
mile = 1760 * yard

# Input
distance = float(input("Distance (in m): "))

# Output
print(f"Distance = {distance} m in various units:")
print(60 * "-")
print(f"{distance / inch:20.2f} {distance / inch:20.2} {"inch":^20}")
print(f"{distance / foot:20.2f} {distance / foot:20.2} {"foot":^20}")
print(f"{distance / yard:20.2f} {distance / yard:20.2} {"yard":^20}")
print(f"{distance / mile:20.2f} {distance / mile:20.2} {"mile":^20}")
print(60 * "-")
```

Listing 15.5 code/2025/ex_2_glorifier.py

```python
# Exercise 2.1: Glorifier

name = input("Please give your name: ")
print(f"All hail {name}")
```

4.4 Decreasing series [Spring, 2024-2025]

4.5 Prime numbers [Spring, 2025-2026]

4.6 Play rock-paper-scissors [Spring, 2025-2026]

4.7 Game of higher-lower [Spring, 2024-2025]

Lists

5.1 Fibonacci sequence

5.2 Printing elements of a list (Spring 2025)

Listing 15.6 code/2025/ex_2_repeat_words.py

```python
# Exercise 2.2: Repeat a word

word = input("Please provide a word: ")
print(8 * (word + " ") + "!")
```

Listing 15.7 code/2025/ex_2_braking_distance.py

```python
# Exercise 2.3: Braking distance
# Calculates the braking distance of a car

# Parameters
tr = 1
g = 9.81
mu_concrete = 0.68
mu_asphalt = 0.72
mu_asphalt_wet = 0.53

v = float(input("Give the speed of the car in km/h: "))
v = v / 3.6
D_concrete = v * tr + v**2 / (2 * mu_concrete * g)
D_asphalt = v * tr + v**2 / (2 * mu_asphalt * g)
D_asphalt_wet = v * tr + v**2 / (2 * mu_asphalt_wet * g)

print(f"Braking distance on dry concrete road: {D_concrete} m")
print(f"Braking distance on dry asphalt road: {D_asphalt} m")
print(f"Braking distance on wet asphalt road: {D_asphalt_wet} m")
```

5.3 List of soccer players (Spring 2025)

5.4 Selling fruit (Spring 2025)

Knuth, Donald E. 1984. "Literate Programming." *Comput. J.* 27 (2): 97–111. https://doi.org/10.1093/comjnl/27.2.97.

Listing 15.8 code/2025/ex_2_radar.py

```python
# Exercise 2.4: Velocity measurement by radar gun
#   Calculate the Doppler beating frequency measured by
#   a police-radar for a car speeding at 200 km/h

# Parameters
c = 3e8
cp = c / 1.0003
v = 200 / 3.6
ft = 24 * 1e9

# Calculation
fd = 2 * v * ft / cp

# Output
print(f"Doppler beating frequency = {fd} Hz")
```

Listing 15.9 code/2025/ex_2_car_tax.py

```python
# Exercise 2.5: Chevaux-Vapeur car tax calculator
# Calculate the car tax in old France

# Parameters
K = 1.5e-4

# Input
C = float(input("Cylinder capacity: "))
omega = float(input("Max revolutions per minute: "))

# Calculate tax
CV = C * K * 60 * omega

# Output
print(f"Tax: {CV}")
```

Listing 15.10 code/2025/ex_2_format_table.py

```python
# Exercise 2.6: Formatted table

# Import libraries
from math import sqrt

# Input
f1 = float(input("Give a first decimal number: "))
f2 = float(input("Give a second decimal number: "))

# Output
print(f"        |  number 1  |  number 2  |    sum    ")
print(f"Number  | {f1:10.2f} | {f2:10.2f} | {f1+f2:10.2f}")
print(f"Squares | {f1**2:10.2f} | {f2**2:10.2f} | {(f1+f2)**2:10.2f}")
print(f"Sqrt    | {sqrt(f1):10.2f} | {sqrt(f2):10.2f} | {sqrt(f1+f2):10.2f}")
```

Listing 15.11 code/2026/ex_3_1.1.py

```python
# Practical 3: Exercise 1.1

# Parameters
username = "amapirate"
password = "greenbeans"
# Input
print("Please provide your login credentials:")
username_given = input("Username: ")
password_given = input("Password: ")
# Algorithm
username_cond = username == username_given
password_cond = password == password_given
if username_cond and password_cond:
    print(f"Login successful.")
else:
    print(f"The username or password is wrong!")
```

Listing 15.12 code/2026/ex_3_1.2.py

```
# Practical 3: Exercise 1.2
from math import sqrt

print(f"--- Roots of the equation ax^2 + bx + c = 0 ---")
a = 2
b = 3
c = -5
a = float(input("Parameter a: "))
b = float(input("Parameter b: "))
c = float(input("Parameter c: "))
D = b**2 - 4 * a * c
if D > 0:
    xm = (-b - sqrt(D)) / (2 * a)
    xp = (-b + sqrt(D)) / (2 * a)
    print(f"The discriminant D = {D} > 0, two roots x = {xm}, {xp}")
elif D == 0:
    x = -b / (2 * a)
    print(f"The discriminant D = {D} = 0, one root x = {x}")
else:
    print(f"The discriminant D = {D} < 0, no real roots!")
```

Listing 15.13 code/2026/ex_3_2.1.py

```
# Practical 3: Exercise 2.1

# Using a while loop
n = 5
while n <= 10:
    print(f"1/{n} = {1/n:.3}")
    n = n + 1

# Using a for loop
for n in range(5, 11):
    print(f"1/{n} = {1/n:.3}")
```

Listing 15.14 code/2026/ex_3_2.2.py

```
# Practical 3: Exercise 2.2
from math import e, factorial, fabs

mindiff = 0.001
e_approx = 0
n = 0
while fabs(e - e_approx) > mindiff:
    e_approx = e_approx + 1 / factorial(n)
    print(f"Euler's e = {e:5.5} approximated to {n+1} terms = {e_approx:5.5}")
    n = n + 1
```

Listing 15.15 code/2025/ex_4_1_printing_numbers.py

```python
# Exercise 1: Printing numbers 1-10

# Using a while loop
print("Print 1-10 using a while loop")
i = 0
while i < 11:
    i += 1
    print(i)

# Using a for loop
print("Print 1-10 using a for loop")
for i in range(1, 11):
    print(i)

# Using a for loop, only printing the odd numbers, step in range(begin, end, step)
print("Print odd numbers 1-10 using a for loop")
for i in range(1, 11, 2):
    print(i)
```

Listing 15.16 code/2025/ex_4_2_series.py

```python
# Exercise 2: Decreasing series of numbers

# Parameters
n = 1

# Loop and output
t = 1 / n**4
while t > 0.0001:
    n += 1
    print(t)
    t = 1 / n**4
```

Listing 15.17 `code/2026/ex_3_2.3.py`

```python
# Practical 3: Exercise 2.3
from math import sqrt, floor

N = int(input("Give a positive integer N: "))
n_max = sqrt(N)
is_prime = True
for n in range(2, floor(n_max) + 1):
    if N % n == 0:
        print(f"The number {N} can be divided by {n}")
        is_prime = False

if is_prime:
    print(f"The number {N} is prime")
else:
    print(f"The number {N} is not prime")
```

Listing 15.18 code/2026/ex_3_2.4.py

```
# Practical 3: Exercise 2.4
from random import randint

c1 = "rock"
c2 = "paper"
c3 = "scissors"
score = 0
choice = 0
while choice != 4:
    print("[1] Rock , [2] Paper, [3] Scissors, or [4] Stop the game")
    choice = int(input("  Choice: "))
    pc_choice = randint(1, 3)
    if choice == pc_choice:
        print("I had the same, Score: {score}")
    if choice == 1:
        if pc_choice == 2:
            print(f"You lose, I had {c2}, Score: {score-1}")
        if pc_choice == 3:
            print(f"You win, I had {c3}, Score: {score+1}")
    if choice == 2:
        if pc_choice == 3:
            print(f"You lose, I had {c3}, Score: {score-1}")
        if pc_choice == 1:
            print(f"You win, I had {c1}, Score: {score+1}")
    if choice == 3:
        if pc_choice == 1:
            print(f"You lose, I had {c1}, Score: {score-1}")
        if pc_choice == 2:
            print(f"You win, I had {c2}, Score: {score+1}")

print("Stopping game!")
```

Listing 15.19 code/2025/ex_4_3_higherlower.py

```
# Exercise 3: Higher-lower game

# import libraries
import random

# Parameters
n_min = 0
n_max = 100
n = random.randint(0, 100)

# Print start of game
print(f"Guess the number [{n_min}, {n_max}]: game started!")

# Loop
m = -1
guess_count = 0
while m != n:
    guess_count += 1
    m = int(input("What is the number? "))
    if m > n:
        print("The number is lower, try again.")
    elif m < n:
        print("The number is higher, try again.")

print(f"Yes, that is the number! Congratulations, you guessed it in {guess_count} guesses.")
```

Listing 15.20 code/2026/ex_4_1.py

```
# Practical 4: Exercise 1
fs = [0, 1]
for i in range(8):
    fs.append(fs[-1] + fs[-2])

print(fs)
```

Listing 15.21 code/2025/ex_4_4_lists.py

```python
# Exercise 4: list of colors

# Define the list
colors = ["yellow", "green", "blue", "red", "white", "orange", "black"]

# a. Print the last element to the screen
print("Last color in the list:")
print(colors[-1])

# b. Print the sublist containing the colors of the Turkish flag
print("List of colors of the Turkish flag:")
print(colors[3:5])

# c. Print the sublist with the colors of the rainbow (ignore black and white)
# and order them according to increasing wavelengths
print([colors[2], colors[1], colors[0], colors[5], colors[3]])

# d. Print all the elements one by one to the screen using a loop.
print("All colors one-by-one")
for color in colors:
    print(color)

# e. Prompt the user to tell his favorite color which is not in the list.
# If not in the list, append it to the list. Otherwise reply that it is already in the list.
print("Adding a favorite color.")
color = input("What is your favorite color? ")
if color in colors:
    print(f"{color} is already in our list.")
else:
    colors.append(color)
    print(f"The new list: {colors}")
```

Listing 15.22 code/2025/ex_4_5_soccer_players.py

```python
# Exercise 5: Soccer players from Kazakhstan national football team

players = ["Maksim", "Abat", "Oralkhan", "Ramazan", "Vyacheslav"]

# a. Print the names of the list to the screen line-by-line by using a for loop.
print("b. Players in order of the list:")
for player in players:
    print(player)

# b. Print the names in reversed order to the screen. Hint: You can use `len()` and `range()`
print("b. Players in reverse order:")
players.reverse()
for player in players:
    print(player)
# Reverse again for the next tasks
players.reverse()

# c. Append the name "Dastan" to the list. Print the list, did it change?
print("c. Append name Dastan:")
players.append("Dastan")
print(players)

# d. Select the second player from the list and print: "[players_name] is the best player!" to
print("d. Cheer for the second player of the list:")
print(f"{players[1]} is the best player!")

# e. Remove player "Abat" from the list.
print("e. Remove player Abat from the list:")
players.remove("Abat")
print(players)
```

Listing 15.23 code/2025/ex_4_6_selling_fruit.py

```
# Exercise 6: Selling fruit

# Initialize the lists of the salesperson on the market and the customers (empty) basket
market = ["banana", "apple", "kiwi", "banana", "banana", "orange", "apple", "kiwi", "strawberr
basket = []

stop = False
while not stop:
    fruit = input("What do you like to buy? ")
    if fruit == "That's all":
        stop = True
    elif fruit not in market:
        print(f"Sorry, today I don't have any {fruit}.")
    else:
        print("Here you go.")
        market.remove(fruit)
        basket.append(fruit)

print(f"You bought following fruit: {basket}")
```